The Journal of Open Source Software

StateCounter: state tracking for composite objects in complex combinatorial libraries

Zhihan Liu[1] **and Justin B. Kinney**[1¶]

1 Cold Spring Harbor Laboratory, Cold Spring Harbor, NY, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

StateCounter is a Python library created to solve a technical problem that can arise in a variety of scientific computing tasks: given the state of a composite object created using complex combinatorial processes, determine the states of each component object. The core computational units in StateCounter are Counters, each of which represents the state of an abstract object. After defining a set of parent Counters, users create child counters using combinatorial operations that include Cartesian products, disjoint unions (i.e., stacking), repetition, slicing, shuffling, splitting, and sampling. Then when a user sets the state of a derived Counter, StateCounter automatically computes the state of all parent counters by traversing of the derived Counter's computational graph in reverse. We developed StateCounter specifically to allow tracking of DNA sequence generation and mutation processes used to create complex DNA sequence libraries. More generally, however, StateCounter addresses a fundamental computational need that is not addressed by existing Python software.

Statement of Need

We encountered the need for StateCounter when creating PoolParty, a Python package for generating complex DNA sequence libraries. Fig. 1 shows one example application–creating a DNA sequence library comprising different types of variants of a cis-regulatory element (CRE). Each variant CRE is created using a specific mutational process applied to a wild-type sequence. The first sequence is the wild-type sequence; no mutations are applied. The next 5 sequences comprise CREs with randomly scattered point mutations. The next 5 sequences comprise CREs containing 5 nt deletions at tiled positions. The next 10 sequences comprise CREs with one of two binding sites inserted at 5 tiled postions. Each one of these sequences is then repeated 2 times and linked to a random 5-mer serving as a barcode. The problem we faced was this: given a value n, how do we compute which mutations to introduce into the sequence and which barcode to attach? This computation is particularly useful for systematically naming the generated sequences.

DRAFT

state	name	seq
0	mut_0.v0	TCCCGACT<cre>GGAAAGCaGGCAaTGAGCACcaAGG</cre>ATTACGG<bc>tggcc</bc>AGATCGGA
1	mut_0.v1	TCCCGACT<cre>GGAAAGCaGGCAaTGAGCACcaAGG</cre>ATTACGG<bc>aaaat</bc>AGATCGGA
2	mut_1.v0	TCCCGACT<cre>GGAAAcCGGGCAGTGAGgACACAta</cre>ATTACGG<bc>gtggt</bc>AGATCGGA
3	mut_1.v1	TCCCGACT<cre>GGAAAcCGGGCAGTGAGgACACAta</cre>ATTACGG<bc>ggggt</bc>AGATCGGA
4	mut_2.v0	TCCCGACT<cre>GGAAAGaGGGCAGaGAGCACAatGG</cre>ATTACGG<bc>ctgac</bc>AGATCGGA
5	mut_2.v1	TCCCGACT<cre>GGAAAGaGGGCAGaGAGCACAatGG</cre>ATTACGG<bc>tgatg</bc>AGATCGGA
6	mut_3.v0	TCCCGACT<cre>GGAAcGCGGGCAGTctaCACACAGG</cre>ATTACGG<bc>taata</bc>AGATCGGA
7	mut_3.v1	TCCCGACT<cre>GGAAcGCGGGCAGTctaCACACAGG</cre>ATTACGG<bc>gaccc</bc>AGATCGGA
8	mut_4.v0	TCCCGACT<cre>GcAAAGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>caaaa</bc>AGATCGGA
9	mut_4.v1	TCCCGACT<cre>GcAAAGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>gggcg</bc>AGATCGGA
10	del_0.v0	TCCCGACT<cre>-----GCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>tcctt</bc>AGATCGGA
11	del_0.v1	TCCCGACT<cre>-----GCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>tcgtg</bc>AGATCGGA
12	del_1.v0	TCCCGACT<cre>GGAAA-----CAGTGAGCACACAGG</cre>ATTACGG<bc>tggct</bc>AGATCGGA
13	del_1.v1	TCCCGACT<cre>GGAAA-----CAGTGAGCACACAGG</cre>ATTACGG<bc>aggtg</bc>AGATCGGA
14	del_2.v0	TCCCGACT<cre>GGAAAGCGGG-----AGCACACAGG</cre>ATTACGG<bc>ccccg</bc>AGATCGGA
15	del_2.v1	TCCCGACT<cre>GGAAAGCGGG-----AGCACACAGG</cre>ATTACGG<bc>tatgc</bc>AGATCGGA
16	del_3.v0	TCCCGACT<cre>GGAAAGCGGGCAGTG-----ACAGG</cre>ATTACGG<bc>ggccg</bc>AGATCGGA
17	del_3.v1	TCCCGACT<cre>GGAAAGCGGGCAGTG-----ACAGG</cre>ATTACGG<bc>ggctc</bc>AGATCGGA
18	del_4.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCAC-----</cre>ATTACGG<bc>ctcag</bc>AGATCGGA
19	del_4.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCAC-----</cre>ATTACGG<bc>gaact</bc>AGATCGGA
20	ins_0.site_0.v0	TCCCGACT<cre>aaaaaGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>ctcat</bc>AGATCGGA
21	ins_0.site_0.v1	TCCCGACT<cre>aaaaaGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>taagc</bc>AGATCGGA
22	ins_0.site_1.v0	TCCCGACT<cre>tttttGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>gatct</bc>AGATCGGA
23	ins_0.site_1.v1	TCCCGACT<cre>tttttGCGGGCAGTGAGCACACAGG</cre>ATTACGG<bc>tgata</bc>AGATCGGA
24	ins_1.site_0.v0	TCCCGACT<cre>GGAAAaaaaaCAGTGAGCACACAGG</cre>ATTACGG<bc>gctat</bc>AGATCGGA
25	ins_1.site_0.v1	TCCCGACT<cre>GGAAAaaaaaCAGTGAGCACACAGG</cre>ATTACGG<bc>aggtc</bc>AGATCGGA
26	ins_1.site_1.v0	TCCCGACT<cre>GGAAAtttttCAGTGAGCACACAGG</cre>ATTACGG<bc>tgtat</bc>AGATCGGA
27	ins_1.site_1.v1	TCCCGACT<cre>GGAAAtttttCAGTGAGCACACAGG</cre>ATTACGG<bc>tacga</bc>AGATCGGA
28	ins_2.site_0.v0	TCCCGACT<cre>GGAAAGCGGGaaaaaAGCACACAGG</cre>ATTACGG<bc>ggttc</bc>AGATCGGA
29	ins_2.site_0.v1	TCCCGACT<cre>GGAAAGCGGGaaaaaAGCACACAGG</cre>ATTACGG<bc>cctac</bc>AGATCGGA
30	ins_2.site_1.v0	TCCCGACT<cre>GGAAAGCGGGtttttAGCACACAGG</cre>ATTACGG<bc>actgc</bc>AGATCGGA
31	ins_2.site_1.v1	TCCCGACT<cre>GGAAAGCGGGtttttAGCACACAGG</cre>ATTACGG<bc>tgtac</bc>AGATCGGA
32	ins_3.site_0.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGaaaaaACAGG</cre>ATTACGG<bc>ttccc</bc>AGATCGGA
33	ins_3.site_0.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGaaaaaACAGG</cre>ATTACGG<bc>gatac</bc>AGATCGGA
34	ins_3.site_1.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGtttttACAGG</cre>ATTACGG<bc>cgggt</bc>AGATCGGA
35	ins_3.site_1.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGtttttACAGG</cre>ATTACGG<bc>taaag</bc>AGATCGGA
36	ins_4.site_0.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCACaaaaa</cre>ATTACGG<bc>ttgtt</bc>AGATCGGA
37	ins_4.site_0.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCACaaaaa</cre>ATTACGG<bc>aatat</bc>AGATCGGA
38	ins_4.site_1.v0	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCACttttt</cre>ATTACGG<bc>ttcag</bc>AGATCGGA
39	ins_4.site_1.v1	TCCCGACT<cre>GGAAAGCGGGCAGTGAGCACttttt</cre>ATTACGG<bc>tctct</bc>AGATCGGA

Figure 1: Figure 1: Sequence Library genereated by PoolParty

The need we encountered here is quite general – how does one trace the state of a complex composite object back to the states of its components. This is the problem that StateCounter was created to solve.

State of the Field

Python provides several tools for working with combinatorial structures, but none address the state propagation problem that StateCounter solves. The `itertools` module provides `product` for Cartesian products and `chain` for concatenation, but these return iterators or tuples without any mechanism to trace a flat index back to component values. NumPy's `unravel_index` and `ravel_multi_index` functions convert between flat and multi-dimensional indices, but only for rectangular array shapes; they cannot represent structures like disjoint unions or nested combinations of products and unions. Manual index arithmetic using `divmod` works for simple products but does not compose—each additional operation (slicing, shuffling, sampling) requires rewriting the index logic from scratch.

Software Design

StateCounter addresses this gap by constructing a directed acyclic graph (DAG) of counters where setting the state of a derived counter automatically propagates to all parent counters. Thus, while users construct the DAG from leaves to root, state information flows from root to leaves. This enables arbitrary composition of products, stacks, slices, shuffles, samples, and splits while maintaining correct state tracking throughout.

The basic unit is a Counter, which represents an object that can take on a finite number of discrete states numbered from 0 to $n-1$. Users first create "leaf" counters specifying the number of states, then compose them using operations to build derived counters. The following operations are supported:

Operation	Description
`product`	Cartesian product; all parent counters active
`stack`	Disjoint union; one parent counter active at a time
`slice`	Select subset of states
`shuffle`	Randomly permute states
`sample`	Sample states with or without replacement
`split`	Divide into sub-counters by count or proportion
`repeat`	Cycle through states multiple times
`sync`	Synchronize counters in lockstep
`interleave`	Alternate between counters

All counters must be created within a `Manager` context that tracks counter relationships and assigns unique identifiers. When a counter appears in multiple branches of the DAG, StateCounter detects conflicting state assignments—situations where two paths would require the same parent counter to have different values simultaneously.

Example

The following code constructs the counter DAG for the sequence library in Fig. 1. Five leaf counters represent the component dimensions: mutation index, deletion position, insertion site, insertion position, and barcode variant. These are composed using `product` and `stack` to build the full 40-state counter.

```python
from statecounter import Manager, Counter, product, stack

# Initialize manager
with Manager() as mgr:

    # Define leaf counters
    mut_counter = Counter(5).named('mut')
    del_counter = Counter(5).named('del')
    ins_site_counter = Counter(2).named('ins_site')
    ins_position_counter = Counter(5).named('ins_position')
    v_counter = Counter(2).named('v')

    # Build composite counters
    ins_counter = product([ins_position_counter, ins_site_counter]).named('ins')
    cre_counter = stack([mut_counter, del_counter, ins_counter]).named('cre')
    seq_counter = product([v_counter, cre_counter]).named('seq')

# Print DAG
seq_counter.print_dag('minimal')
```

DRAFT

The `print_dag()` method displays the counter hierarchy. The `cre` counter stacks three alternatives (5 mutations + 5 deletions + 10 insertions = 20 states), and `seq` takes the product with the 2-state barcode counter, yielding 40 total states.

```
seq (n=40)
└── [Product]
    ├── v (n=2)
    └── cre (n=20)
        └── [Stack]
            ├── mut (n=5)
            ├── del (n=5)
            └── ins (n=10)
                └── [Product]
                    ├── ins_position (n=5)
                    └── ins_site (n=2)
```

DRAFT

	name	v.state	cre.state	mut.state	del.state	ins.state	ins_position.state	ins_site.state
0	mut_0.v0	0	0	0	None	None	None	None
1	mut_0.v1	1	0	0	None	None	None	None
2	mut_1.v0	0	1	1	None	None	None	None
3	mut_1.v1	1	1	1	None	None	None	None
4	mut_2.v0	0	2	2	None	None	None	None
5	mut_2.v1	1	2	2	None	None	None	None
6	mut_3.v0	0	3	3	None	None	None	None
7	mut_3.v1	1	3	3	None	None	None	None
8	mut_4.v0	0	4	4	None	None	None	None
9	mut_4.v1	1	4	4	None	None	None	None
10	del_0.v0	0	5	None	0	None	None	None
11	del_0.v1	1	5	None	0	None	None	None
12	del_1.v0	0	6	None	1	None	None	None
13	del_1.v1	1	6	None	1	None	None	None
14	del_2.v0	0	7	None	2	None	None	None
15	del_2.v1	1	7	None	2	None	None	None
16	del_3.v0	0	8	None	3	None	None	None
17	del_3.v1	1	8	None	3	None	None	None
18	del_4.v0	0	9	None	4	None	None	None
19	del_4.v1	1	9	None	4	None	None	None
20	ins_0.site_0.v0	0	10	None	None	0	0	0
21	ins_0.site_0.v1	1	10	None	None	0	0	0
22	ins_0.site_1.v0	0	11	None	None	1	0	1
23	ins_0.site_1.v1	1	11	None	None	1	0	1
24	ins_1.site_0.v0	0	12	None	None	2	1	0
25	ins_1.site_0.v1	1	12	None	None	2	1	0
26	ins_1.site_1.v0	0	13	None	None	3	1	1
27	ins_1.site_1.v1	1	13	None	None	3	1	1
28	ins_2.site_0.v0	0	14	None	None	4	2	0
29	ins_2.site_0.v1	1	14	None	None	4	2	0
30	ins_2.site_1.v0	0	15	None	None	5	2	1
31	ins_2.site_1.v1	1	15	None	None	5	2	1
32	ins_3.site_0.v0	0	16	None	None	6	3	0
33	ins_3.site_0.v1	1	16	None	None	6	3	0
34	ins_3.site_1.v0	0	17	None	None	7	3	1
35	ins_3.site_1.v1	1	17	None	None	7	3	1
36	ins_4.site_0.v0	0	18	None	None	8	4	0
37	ins_4.site_0.v1	1	18	None	None	8	4	0
38	ins_4.site_1.v0	0	19	None	None	9	4	1
39	ins_4.site_1.v1	1	19	None	None	9	4	1

Figure 2: Figure 2: State tracking carried out by StateCounter

Research Impact Statement

AI Usage Disclosure

Z.L. and J.B.K. concieved the project and designed the software architecture. Code was written in Cursor by J.B.K. with iterative assisstance from Claude Opus 4.5. Tests and documentation were written by Opus 4.5 with human editing. Z.H. and J.B.K. wrote the paper using Claude Opus 4.5 for help with the initial draft. Z.L. and J.B.K. bear full responsibility for the text of the paper, the documentation, and the content of the software.

Acknowledgements

This work was supported by NIH grant R35 GM148235.

References

DRAFT